OMWAY TECHNOLOGIES

Technical Assignment

Assignment: Stock Market Application
Position: Full Stack Developer / Python (AI-ML) Engineer — Experienced
Time Allotted: 1 Day (6–8 Hours)
Submission Format: GitHub repository link + Live URL (if deployed)

Objective

Build a stock market application: crawl news and one month of trading data for 5–10 companies, automatically categorize news by company, analyze buyer/seller behavior, and serve it through a role-gated system with live updates. Design the architecture yourself — this brief states the required features and constraints, not the implementation.

1. Data Crawling & Collection

- News crawlers (Scrapy or equivalent) covering multiple portals: merolagani.com, sharesansar.com, nepsealpha.com, bizmandu.com — headline, body/excerpt, date, source, URL. Deduplicate by URL, respect robots.txt.
- NEPSE trading data for 5–10 companies: one month of daily OHLCV, plus floorsheet-level transactions (buyer/seller broker, quantity, rate) for a sample of days where reachable.
- Crawl jobs distributed through a message queue with per-portal rate limiting and a scheduler — not a single script run by hand.

2. Automatic News Categorization

- Multi-label tagging of each article by company, with a confidence score.
- Any approach is acceptable (keyword/NER/embeddings/LLM) — justify your choice in the README.
- An Analyst role can correct mis-tagged articles; log corrections for future retraining.

3. Buyer & Seller Behavior Analysis

- VWAP vs. closing price, and a buy/sell pressure indicator from price/volume movement.
- Volume anomaly detection (state your threshold and method).
- Broker-level buy/sell breakdown from floorsheet data, where collected.
- A simple check of whether news activity lines up with next-day price/volume movement.
- A short written summary of the most interesting pattern found.

4. Backend & Real-Time Requirements

- Python API (FastAPI or Django REST Framework) plus a WebSocket/SSE channel that pushes updates to the dashboard when new data lands.
- Time-series-aware storage for price data (e.g. TimescaleDB or an equivalent partitioning approach).
- A queue (Redis/Celery or equivalent) for crawling and categorization inference.
- Centralized validation and error handling; API documented (OpenAPI/Swagger or Postman).

5. Frontend Dashboard (React / Next.js)

- Per-company view: price/volume chart, categorized news feed, behavior analysis — updating live, not only on page load.
- Cross-company comparison view (most volatile, most active, most in-the-news).

- Admin screen for crawl monitoring and news-category correction.
- Responsive on desktop and tablet.

6. Role-Based Access Control

Role	Permissions
Admin	Manage watchlist, trigger/monitor crawl runs, manage users
Analyst	View all data, correct categorization, export reports
Viewer	Read-only dashboards and reports

Enforce every role check server-side.

7. API Surface (indicative — extend as needed)

Companies & Market Data

Method	Endpoint	Description
GET	/api/companies	Watchlist of tracked companies
GET	/api/companies/:id/prices?range=30d	Daily OHLCV for a company

News & Analysis

Method	Endpoint	Description
GET	/api/news?company_id=	Categorized news for a company
POST	/api/news/:id/recategorize	Analyst correction (role-gated)
GET	/api/companies/:id/behavior-summary	VWAP, pressure indicator, anomalies

Admin & Real-Time

Method	Endpoint	Description
POST	/api/admin/crawl-runs	Trigger a crawl run (role-gated: Admin)
WS	/ws/updates	Push new results to connected dashboards

Non-Functional Expectations

- Handles 5–10 companies and a month of data without manual babysitting; adding a company is a config change, not a code change.
- Failures (crawl or API) are logged and surfaced in the Admin view, not just in console output.
- No hard-coded secrets — use environment variables or a config file.

Deliverables

- GitHub repository with a clear commit history.
- README: setup, data sources, categorization approach and trade-offs, assumptions made under the time limit.
- Live URL if deployed, or docker-compose for the full stack locally. Otherwise make a video presentation
- The written findings summary from Section 3.

Evaluation Criteria

Area	Weight	Looking for
Crawling & pipeline reliability	20%	Queue-based, rate-limited, runs unattended
News categorization quality	15%	Sound method, multi-label, justified
Behavior analysis & insight	15%	Real, defensible insight
Backend architecture & RBAC	20%	Clean design, enforced roles
Real-time dashboard & UX	15%	Live updates work, usable
Documentation	15%	README explains real decisions

Bonus (Optional)

- Next-day price-direction classifier (exploratory, not production-grade).
- CI pipeline running lint and tests.
- Alerting when a volume anomaly threshold is crossed.

Prepared by: Omway Technologies